

	SR UNIVERSITY	
	Campus Warangal	
	Program: II - B. Tech (CS& AI) Batch: 16	
	Name: R.SAIMANASA	Reg. No:16
	Department: Computer Science and AI	Semester: 1
	AI Assisted Coding-Assignment	Date: SEP /22/2026

1. Question:

– AI-Assisted Bubble Sort Analysis

Problem / Starter Code:

```
numbers = [64, 34, 25, 12, 22, 11, 90]
```

```
# TODO:
```

```
# Implement Bubble Sort in ascending order.
```

```
# Count comparisons and swaps.
```

```
# Display the sorted list.
```

Student Tasks:

- Implement Bubble Sort.
- Use AI to explain one complete pass of Bubble Sort.
- Count comparisons and swaps.
- Test at least three input lists.
- Ask AI about an early-exit optimisation.
- Compare the original and improved versions.

2. Prompt:

“Explain Bubble Sort step by step for the given list. Identify opportunities to reduce unnecessary comparisons. Do not write the complete program yet

.

3. Code:

```
ai asss.py > ...
1  def bubble_sort(values):
2      n = len(values)
3
4      for pass_number in range(n - 1):
5          swapped = False
6
7          # The largest unsorted value moves to the end during each pass.
8          for index in range(n - 1 - pass_number):
9              if values[index] > values[index + 1]:
10                 values[index], values[index + 1] = values[index + 1], values[index]
11                 swapped = True
12
13             # Stop early when the list is already sorted.
14             if not swapped:
15                 break
16
17     return values
18
19
20 numbers = [5, 1, 4, 2, 8]
21 print("Before sorting:", numbers)
22 print("After sorting:", bubble_sort(numbers))
```

4. Output:

```
PS C:\Users\saima\OneDrive\Desktop\aicoding\aicoding> &
C:\Users\saima\AppData\Local\Programs\Python\Python314\python.exe
"c:/Users/saima/OneDrive/Desktop/aicoding/aicoding/ai asss.py"
Before sorting: [5, 1, 4, 2, 8]
After sorting: [1, 2, 4, 5, 8]
PS C:\Users\saima\OneDrive\Desktop\aicoding\aicoding> &
C:\Users\saima\AppData\Local\Programs\Python\Python314\python.exe
"c:/Users/saima/OneDrive/Desktop/aicoding/aicoding/ai asss.py"
Before sorting: [5, 1, 4, 2, 8]
After sorting: [1, 2, 4, 5, 8]
PS C:\Users\saima\OneDrive\Desktop\aicoding\aicoding>
```

5.Observation: Bubble Sort arranges the elements in ascending order by repeatedly swapping adjacent elements.

1. The given list [5, 1, 4, 2, 8] is sorted as [1, 2, 4, 5, 8].
2. The swapped flag improves efficiency by stopping early if the list is already sorted.

2 Question:

Linear Search vs Binary Search

Problem / Starter Code:

```
numbers = [5, 12, 18, 23, 31, 45, 56, 72, 89]
```

```
target = 45
```

```
# TODO:
```

```
# Implement Linear Search.
```

```
# Implement Binary Search.
```

```
# Compare the number of comparisons.
```

Student Tasks:

- Implement Linear Search and Binary Search.
- Use AI to explain why Binary Search requires sorted data.
- Count comparisons for both algorithms.
- Test first, middle, last and absent elements.
- Compare the results and explain suitable use cases.
- Use AI to suggest a valid optimisation.

prompt:

Write a Python program to implement Linear Search and Binary Search for the given sorted list. Count and compare the number of comparisons for both algorithms. Test first, middle, last, and absent elements, explain why Binary Search needs sorted data, compare their use cases, and suggest one valid optimization using

code:

```
def linear_search(values, target):  
    """Return the target index and number of comparisons."""  
    comparisons = 0  
  
    for index, value in enumerate(values):  
        comparisons += 1  
        if value == target:  
            return index, comparisons  
    return -1, comparisons  
  
def binary_search(values, target):  
    """Return the target index and number of comparisons.  
    The values list must be sorted in ascending order.  
    """  
    left = 0  
    right = len(values) - 1  
    comparisons = 0  
    while left <= right:  
        middle = (left + right) // 2  
        comparisons += 1  
        if values[middle] == target:  
            return middle, comparisons  
        elif values[middle] < target:  
            left = middle + 1  
        else:  
            right = middle - 1  
    return -1, comparisons  
  
def print_result(algorithm, result):  
    index, comparisons = result  
    location = index if index != -1 else "absent"  
    print(f"{algorithm}: position = {location}, comparisons = {comparisons}")# Binary Search requires this  
list to be sorted.numbers = [3, 7, 11, 18, 24, 31, 42, 50]
```

```

targets = {
    "first": numbers[0],
    "middle": numbers[len(numbers) // 2],
    "last": numbers[-1],
    "absent": 99,
}

print("Sorted list:", numbers)
print("\nComparison results:")
for test_name, target in targets.items():
    print(f"\n{test_name.title()} element ({target}):")
    print_result("Linear Search", linear_search(numbers, target))
    print_result("Binary Search", binary_search(numbers, target))
print("\nExplanation:")
print("Binary Search needs sorted data because it discards half of the")
print("remaining list after each comparison. Without ordering, the discarded")
print("half might still contain the target.")
print("\nUse cases:")
print("Linear Search is suitable for small or unsorted lists.")
print("Binary Search is suitable for large sorted lists and repeated searches.")
print("\nAI-suggested optimization:")
print("Keep the list sorted when many searches are needed and use Binary Search.")
print("This reduces the search time from O(n) to O(log n) per search.")

```

Output:

PS C:\Users\saima\OneDrive\Desktop\aicoding\aicoding> &
C:\Users\saima\AppData\Local\Programs\Python\Python314\python.exe
c:/Users/saima/OneDrive/Desktop/aicoding/aicoding/aiasss.py

File "c:\Users\saima\OneDrive\Desktop\aicoding\aicoding\aiasss.py", line 70

print("This reduces the search time from O(n) to O(log n) per search.

SyntaxError: unterminated string literal (detected at line 70)

PS C:\Users\saima\OneDrive\Desktop\aicoding\aicoding>

Observation:

Linear Search checks elements sequentially, so comparisons increase with the target's position. Binary Search repeatedly halves the search range, requiring fewer comparisons on sorted lists. Therefore, Linear Search suits small or unsorted data, while Binary Search is better for large sorted data.

3 Question:

Compare Sorting Algorithms

Problem / Starter Code:

```
data = [45, 12, 78, 34, 23, 89, 5, 67, 10, 56]
```

TODO:

Sort using Bubble Sort, Selection Sort and Insertion Sort.

Compare their performance.

Student Tasks:

- Implement all three sorting algorithms.
- Use the same input for each algorithm.
- Count comparisons and swaps/shifts.
- Use AI to explain strengths and weaknesses.
- Ask AI which algorithm suits nearly sorted data.
- Compare experimental results with theoretical complexity.

Prompt:

Write Python code that compares Bubble Sort, Selection Sort, and Insertion Sort on this dataset:

python

```
data = [45, 12, 78, 34, 23, 89, 5, 67, 10, 56]
```

Requiriments:

Implement all three algorithms from scratch (no built-in `sort()`).

Each implementation must use a **copy** of the original list so all three sort the same starting data.

Inside each algorithm, count the number of **comparisons** and **swaps/shifts** made, and print these counts after sorting.

Print the sorted output for each algorithm to confirm correctness.

Print a summary table comparing comparisons and swaps across all three algorithms.

Add brief comments explaining each algorithm's logic.

After the code, explain in plain language:

The strengths and weaknesses of each algorithm.

Which algorithm performs best on **nearly sorted data**, and why.

How the experimental comparison/swap counts relate to their theoretical time complexities ($O(n^2)$ worst case, etc.).

Code:

```
data = [45, 12, 78, 34, 23, 89, 5, 67, 10, 56]

def bubble_sort(values):
    """Sort by repeatedly swapping adjacent out-of-order values."""
    numbers = values.copy()
    comparisons = 0
    swaps = 0
    for end in range(len(numbers) - 1, 0, -1):
        for index in range(end):
            comparisons += 1
            if numbers[index] > numbers[index + 1]:
                numbers[index], numbers[index + 1] = (
```

```

        numbers[index + 1],
        numbers[index],
    )

    swaps += 1

    return numbers, comparisons, swaps
def selection_sort(values):
    """Find the smallest remaining value and place it in the next position."""
    numbers = values.copy()
    comparisons = 0
    swaps = 0
    for position in range(len(numbers) - 1):
        smallest_index = position
        for index in range(position + 1, len(numbers)):
            comparisons += 1
            if numbers[index] < numbers[smallest_index]:
                smallest_index = index
        if smallest_index != position:
            numbers[position], numbers[smallest_index] = (
                numbers[smallest_index],
                numbers[position],
            )
            swaps += 1
    return numbers, comparisons, swaps
def insertion_sort(values):
    """Insert each value into its correct position in the sorted prefix."""
    numbers = values.copy()
    comparisons = 0
    shifts = 0
    for index in range(1, len(numbers)):
        key = numbers[index]
        previous = index - 1
        while previous >= 0:
            comparisons += 1

```

```

        if numbers[previous] <= key:
            break

        numbers[previous + 1] = numbers[previous]

        shifts += 1

        previous -= 1

    numbers[previous + 1] = key

    return numbers, comparisons, shifts

bubble_result = bubble_sort(data)
selection_result = selection_sort(data)
insertion_result = insertion_sort(data)

results = [

    ("Bubble Sort", bubble_result),

    ("Selection Sort", selection_result),

    ("Insertion Sort", insertion_result),

]

for name, (sorted_data, comparisons, swaps_or_shifts) in results:

    print(f"{name}:")

    print(f"  Sorted output: {sorted_data}")

    print(f"  Comparisons: {comparisons}")

    print(f"  Swaps/shifts: {swaps_or_shifts}")

    print()

print("Summary:")

print(f"{'Algorithm':<18}{'Comparisons':>14}{'Swaps/shifts':>15}")

print("-" * 47)

for name, (_, comparisons, swaps_or_shifts) in results:

    print(f"{name:<18}{comparisons:>14}{swaps_or_shifts:>15}")

```

Output:

Sorted output: [5, 10, 12, 23, 34, 45, 56, 67, 78, 89]

Comparisons: 31

Swaps/shifts: 24

Summary:

Algorithm	Comparisons	Swaps/shifts
-----------	-------------	--------------

Bubble Sort	45	24
-------------	----	----

Selection Sort	45	6
----------------	----	---

Insertion Sort	31	24
----------------	----	----

PS C:\Users\saima\OneDrive\Desktop\aicoding\aicoding>

Observation:

1. **Selection Sort** made the fewest swaps but the most comparisons overall, since it always scans the full remaining list even if the array is nearly sorted.
2. **Bubble Sort** performed the most comparisons and swaps among the three, showing it is the least efficient for random or unsorted data of this size.
3. **Insertion Sort** performed noticeably fewer comparisons and shifts than the others, confirming it is the most efficient of the three — especially suited for nearly sorted data.

4 Question:

Optimising a Student Search and Sort Program

Problem / Starter Code:

```
students = [  
    ("Arun", 82),  
    ("Bala", 65),  
    ("Chitra", 91),  
    ("Divya", 74),
```

```
("Esha", 88)  
]
```

```
# TODO:  
# Find the highest mark.  
# Sort students by mark.  
# Search for a student by name.  
# Improve the solution using AI.
```

Student Tasks:

- Write functions for highest mark, sorting and name search.
- Create a straightforward solution first.
- Use AI to identify unnecessary operations.
- Ask AI for a more efficient approach.
- Implement only suggestions you understand.
- Compare original and improved time complexity.

Prompt:

Write a complete Python script (to run in VS Code) that does the following:

1. Uses this dataset:

```
python
```

```
data = [45, 12, 78, 34, 23, 89, 5, 67, 10, 56]
```

1. Implements **Bubble Sort**, **Selection Sort**, and **Insertion Sort** from scratch (no built-in `sort()`), each sorting a separate copy of the original list so the original data stays unchanged.
2. Tracks and prints the number of **comparisons** and **swaps/shifts** for each algorithm as it runs.
3. After sorting, prints a clean summary table comparing all three algorithms side by side, with columns: `Algorithm | Comparisons | Swaps/Shifts``.
4. After the table, prints these three observations as output:

- Selection Sort made the fewest swaps but the most comparisons overall, since it always scans the full remaining list even if the array is nearly sorted.
 - Bubble Sort performed the most comparisons and swaps among the three, showing it is the least efficient for random or unsorted data of this size.
 - Insertion Sort performed noticeably fewer comparisons and shifts than the others, confirming it is the most efficient of the three — especially suited for nearly sorted data.
5. Make sure the code is well-commented, uses clear function names for each sorting algorithm, and is formatted so it runs directly in VS Code without errors. give me code

Code:

```
"""Compare Bubble Sort, Selection Sort, and Insertion Sort."""

def bubble_sort(values):
    """Sort a list with Bubble Sort and return comparisons and swaps."""
    comparisons = 0
    swaps = 0
    length = len(values)
    for pass_number in range(length - 1):
        for index in range(length - 1 - pass_number):
            comparisons += 1
            if values[index] > values[index + 1]:
                values[index], values[index + 1] = values[index + 1], values[index]
                swaps += 1
    return comparisons, swaps

def selection_sort(values):
    """Sort a list with Selection Sort and return comparisons and swaps."""
    comparisons = 0
    swaps = 0
    length = len(values)
    for start_index in range(length - 1):
        smallest_index = start_index
        for index in range(start_index + 1, length):
```

```

        comparisons += 1

        if values[index] < values[smallest_index]:

            smallest_index = index

    if smallest_index != start_index:

        values[start_index], values[smallest_index] = (

            values[smallest_index],

            values[start_index],

        )

        swaps += 1

    return comparisons, swaps
def insertion_sort(values):

    """Sort a list with Insertion Sort and return comparisons and shifts."""

    comparisons = 0

    shifts = 0

```

```

    for index in range(1, len(values)):

        current_value = values[index]

        position = index

        while position > 0:

            comparisons += 1

            if values[position - 1] <= current_value:

                break

            values[position] = values[position - 1]

            shifts += 1

            position -= 1

        values[position] = current_value

    return comparisons, shifts
def print_summary(results):

    """Print sorting statistics in a simple table."""

    print("\nSummary")

    print("-" * 43)

    print(f"{'Algorithm':<18} | {'Comparisons':>11} | {'Swaps/Shifts':>12}")

    print("-" * 43)

```

```

    for algorithm, comparisons, swaps_or_shifts in results:
        print(f"{algorithm:<18} | {comparisons:>11} | {swaps_or_shifts:>12}")

    print("-" * 43)
def main():
    # Keep the original data unchanged by giving each algorithm its own copy.
    data = [45, 12, 78, 34, 23, 89, 5, 67, 10, 56]

    bubble_values = data.copy()
    selection_values = data.copy()
    insertion_values = data.copy()

    bubble_comparisons, bubble_swaps = bubble_sort(bubble_values)
    selection_comparisons, selection_swaps = selection_sort(selection_values)
    insertion_comparisons, insertion_shifts = insertion_sort(insertion_values)

    print("Original data: ", data)
    print("Bubble Sort:   ", bubble_values)
    print("Selection Sort:", selection_values)
    print("Insertion Sort:", insertion_values)

    results = [
        ("Bubble Sort", bubble_comparisons, bubble_swaps),
        ("Selection Sort", selection_comparisons, selection_swaps),
        ("Insertion Sort", insertion_comparisons, insertion_shifts),
    ]

    print_summary(results)

    print("\nObservations")

    print("- Selection Sort made the fewest swaps but the most comparisons overall, since it always scans
the full remaining list even if the array is nearly sorted.")

    print("- Bubble Sort performed the most comparisons and swaps among the three, showing it is the least
efficient for random or unsorted data of this size.")

    print("- Insertion Sort performed noticeably fewer comparisons and shifts than the others, confirming
it is the most efficient of the three - especially suited for nearly sorted data.")

```

Output:

Insertion Sort | 31 | 24

Observations

- Selection Sort made the fewest swaps but the most comparisons overall, since it always scans the full remaining list even if the array is nearly sorted.
- Bubble Sort performed the most comparisons and swaps among the three, showing it is the least efficient for random or unsorted data of this size.
- Insertion Sort performed noticeably fewer comparisons and shifts than the others, confirming it is the most efficient of the three - especially suited for nearly sorted data.

PS C:\Users\saima\OneDrive\Desktop\aicoding\aicoding>

Observation:

- **Insertion Sort** was the most efficient overall — fewest comparisons (31) among the three.
- **Selection Sort** had the fewest swaps (6) but the most comparisons (45), since it always scans the full remaining list.
- **Bubble Sort** tied Insertion Sort on swaps (24) but needed more comparisons (44) than Insertion Sort, making it the least efficient of the three here.

5 Question:

– Optimisation Challenge: Best Combination

Problem / Starter Code:

```
items = [  
    ("Laptop", 60000),  
    ("Phone", 30000),  
    ("Tablet", 20000),  
    ("Watch", 10000),  
    ("Headphones", 5000)  
]
```

```
budget = 50000
```

```
# TODO:
```

```
# Find a combination of items whose total price
```

```
# is as high as possible without exceeding the budget.
```

Student Tasks:

- Design a solution to maximise total value without exceeding the budget.
- Start with a simple approach and explain it.
- Use AI to compare Brute Force, Greedy and Dynamic Programming

Prompt:

Write a complete Python script (to run in VS Code) that does the following:

1. Uses this dataset:

```
python
```

```
'''
```

```
data = [45, 12, 78, 34, 23, 89, 5, 67, 10, 56]
```

```
'''
```

1. Implements **Bubble Sort**, **Selection Sort**, and **Insertion Sort** from scratch (no built-in `sort()`), each sorting a separate copy of the original list so the original data stays unchanged.
2. Tracks and prints the number of **comparisons** and **swaps/shifts** for each algorithm as it runs.
3. After sorting, prints a clean summary table comparing all three algorithms side by side, with columns: ``Algorithm | Comparisons | Swaps/Shifts``.
4. After the table, prints these three observations as output:

- Selection Sort made the fewest swaps but the most comparisons overall, since it always scans the full remaining list even if the array is nearly sorted.
- Bubble Sort performed the most comparisons and swaps among the three, showing it is the least efficient for random or unsorted data of this size.
- Insertion Sort performed noticeably fewer comparisons and shifts than the others, confirming it is the most efficient of the three — especially suited for nearly sorted data.

4. Make sure the code is well-commented, uses clear function names for each sorting algorithm, and is formatted so it runs directly in VS Code without errors. give me code

Code:

```
from itertools import combinations

# Each tuple contains an item name and its price/value.
items = [
    ("Laptop", 60000),
    ("Phone", 30000),
    ("Tablet", 20000),
    ("Watch", 10000),
    ("Headphones", 5000),
]
budget = 50000

def brute_force_knapsack(items, budget):
    """Try every possible subset and return the valid subset with greatest value.
    Time complexity:  $O(2^n * n)$ , because there are  $2^n$  subsets and calculating
    each subset's total takes up to  $n$  operations.
    Space complexity:  $O(n)$  for the stored best subset, excluding combinations.
    """
    best_items = []
    best_total = 0
    # A subset can contain any number of items from 0 through  $n$ .
    for subset_size in range(len(items) + 1):
        for subset in combinations(items, subset_size):
            total = sum(price for _, price in subset)
            if total <= budget and total > best_total:
                best_items = list(subset)
                best_total = total
    return best_items, best_total

def greedy_knapsack(items, budget):
    """Pick the most expensive item that still fits at each step.
    Time complexity:  $O(n \log n)$  because the items are sorted.
    Space complexity:  $O(n)$  for the sorted copy and selected items.
    This is a quick heuristic, but it does not guarantee the optimum for 0/1
    knapsack because choosing a large item can block a better combination.
    """
    selected_items = []
    total = 0
    # Sorting by price is sensible when the goal is to spend as much as possible.
    for item in sorted(items, key=lambda item: item[1], reverse=True):
        name, price = item
        if total + price <= budget:
            selected_items.append(item)
            total += price
    return selected_items, total

def dynamic_programming_knapsack(items, budget):
    """Solve 0/1 knapsack with a budget-indexed table and reconstruct the items.
    Time complexity:  $O(n * \text{budget})$ .
    Space complexity:  $O(n * \text{budget})$  for the DP table.
    """
    item_count = len(items)
    # dp[i][current_budget] is the greatest value using the first i items.
    dp = [[0] * (budget + 1) for _ in range(item_count + 1)]
    for i in range(1, item_count + 1):
```

```

_, price = items[i - 1]
for current_budget in range(budget + 1):
    # Option 1: do not choose this item.
    dp[i][current_budget] = dp[i - 1][current_budget]
    # Option 2: choose this item if it fits, but only once.
    if price <= current_budget:
        with_item = dp[i - 1][current_budget - price] + price
        dp[i][current_budget] = max(dp[i][current_budget], with_item)
# Walk backward through the table to identify the chosen items.
selected_items = []
current_budget = budget
for i in range(item_count, 0, -1):
    if dp[i][current_budget] != dp[i - 1][current_budget]:
        item = items[i - 1]
        selected_items.append(item)
        current_budget -= item[1]
selected_items.reverse()
return selected_items, dp[item_count][budget]
def print_result(approach, selected_items, total, explanation):
    """Print one algorithm's result in a consistent format."""
    print(f"{approach}")
    print("-" * len(approach))
    print("Selected items:")
    for name, price in selected_items:
        print(f"  - {name}: {price:,}")
    print(f"Total price: {total:,}")
    print(f"Explanation: {explanation}")
    print()
def main():
    # Run each strategy independently using the same original input data.
    brute_items, brute_total = brute_force_knapsack(items, budget)
    greedy_items, greedy_total = greedy_knapsack(items, budget)
    dp_items, dp_total = dynamic_programming_knapsack(items, budget)
    print(f"Budget: {budget:,}\n")
    print_result(
        "1. Brute Force",
        brute_items,
        brute_total,
        "Good for small item lists when correctness is essential; bad for large "
        "lists because the number of subsets grows exponentially.",
    )
    print_result(
        "2. Greedy",
        greedy_items,
        greedy_total,
        "Good when a fast heuristic is acceptable; bad for general 0/1 knapsack "
        "because it does not guarantee the best combination.",
    )
    print_result(
        "3. Dynamic Programming",
        dp_items,
        dp_total,
        "Good when the budget is a manageable integer and the optimal answer is "
        "required; bad when the budget is extremely large because the table grows.",
    )
    # Example of greedy's limitation: with budget 10 and items worth 6, 5, 5,
    # greedy chooses 6, while the optimal choice is 5 + 5 = 10.
    print("Greedy limitation example:")
    print("With budget 10 and prices [6, 5, 5], greedy chooses 6, but 5 + 5 is optimal.")
    print()
    print("Comparison")

```

```

print("=" * 72)
print(f"{'Approach':<22} {'Time Complexity':<20} {'Space Complexity':<18} {'Optimal?'}")
print("-" * 72)
print(f"{'Brute Force':<22} {'O(2^n * n)':<20} {'O(n)':<18} Yes")
print(f"{'Greedy':<22} {'O(n log n)':<20} {'O(n)':<18} No")
print(f"{'Dynamic Programming':<22} {'O(n * budget)':<20} {'O(n * budget)':<18} Yes")
if __name__ == "__main__":
    main()

```

Output:

Greedy limitation example:

With budget 10 and prices [6, 5, 5], greedy chooses 6, but 5 + 5 is optimal.

Comparison

```

=====
=====

```

Approach	Time Complexity	Space Complexity	Optimal?
----------	-----------------	------------------	----------

```

-----

```

Brute Force	$O(2^n * n)$	$O(n)$	Yes
-------------	--------------	--------	-----

Greedy	$O(n \log n)$	$O(n)$	No
--------	---------------	--------	----

Dynamic Programming	$O(n * \text{budget})$	$O(n * \text{budget})$	Yes
---------------------	------------------------	------------------------	-----

PS C:\Users\saima\OneDrive\Desktop\aicoding\aicoding>

Observation:

1. Brute Force finds the optimal solution (Phone + Tablet = ₹50,000) but is slow for large inputs ($O(2^n)$).
2. Greedy is fast but not always optimal — it can pick items that waste budget or miss better combinations.
3. Dynamic Programming gives the same optimal result as Brute Force but much faster ($O(n \times \text{budget})$), making it the best overall choice.

